Проход по использованию

Английский оригинал: USAGE.md.

Один сценарий от начала до конца: собрать окружение, сгенерировать тестовые данные, открыть захват, применить интерпретацию и получить отчёт. Каждый шаг перечисляет команду и то, что она должна произвести. Пути заданы относительно корня репозитория.

У инструмента три стадии, каждая пригодна сама по себе:

- `src.capture` — на входе PCAP/PCAPNG, на выходе нормализованный захват (сессии, направленные потоки, провенанс байт).
- `src.protocol` — декларативное правило, применённое к нормализованному захвату, дающее результаты и статусы по каждому сообщению.
- `src.project` — переносной каталог проекта, сквозной конвейер и отчёт.

1. Окружение

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e ".[dev]"
```

Ожидается: пакет ставится и `pytest` становится доступен. Python 3.12 или новее, Linux x86-64.

2. Тестовые данные

Корпус закоммичен в `tests/corpus/`. Пересобирайте его только если меняете генераторы:

```
python -m scripts.generate_corpus
```

Ожидается: пишутся `tests/corpus/corpus_capture_01.pcapng`, `corpus_capture_02.pcapng`, `corpus_capture_defects.pcapng`, `synthetic_live.pcapng` и их журналы. Эталонные выгрузки в `tests/corpus/reference_export/` от чистого прогона не меняются; `git status` должен остаться чистым.

3. Нормализовать захват

```
python -m src.capture.cli \
  --pcap tests/corpus/corpus_capture_01.pcapng \
  --out out/cap01.json \
  --source-name captures/corpus_capture_01.pcapng
```

Ожидается: однострочная JSON-сводка в stdout с `capture_id`, `sessions` и `diagnostics`; `out/cap01.json` держит нормализованный захват с, для каждой сессии и направления, `bytes_b64`, `provenance` и `diagnostics`.

Добавьте `--stream`, чтобы обработать большой захват блоками, или `--verify-checksums`, чтобы сообщать несовпадения контрольных сумм TCP вместо игнорирования. Добавьте `--wireshark-`

`pcap out/reassembled.pcap`, чтобы записать пересобранные потоки как pcap для просмотра в Wireshark.

4. Сквозной конвейер

Одна команда проводит захват в проект, нормализованный захват, необязательный результат и отчёт:

```
python -m src.project.cli pipeline \
  --pcap tests/corpus/corpus_capture_01.pcapng \
  --project project.madrigal \
  --report reports/REPORT.md
```

Ожидается: JSON-сводка, перечисляющая проект, нормализованный захват, число сессий и пути отчётов. Каталог проекта выглядит так:

```
project.madrigal/
  manifest.json
  captures/corpus_capture_01.pcapng
  results/normalized.json
  reports/REPORT.md
  reports/REPORT.html
  logs/  rules/  annotations.sqlite
```

Без правила отчёт прямо говорит, что правило не применялось. Добавьте правило, чтобы описать структуру сообщения:

```
python -m src.project.cli pipeline \
  --pcap tests/corpus/corpus_capture_01.pcapng \
  --project project.madrigal \
  --rule src/protocol/examples/set_parameter_request.json \
  --force
```

Ожидается: появляется `results/result.json` с `status` по каждому сообщению и `summary` счётчиков статусов, а отчёт получает гипотезу, список контрпримеров и открытые вопросы. Контрпримеры — сообщения, которые правило не смогло подтвердить; каждый привязан к смещению в потоке.

5. Управление проектом

```
python -m src.project.cli create --path /tmp/lab/project.madrigal --name "parameter study"
python -m src.project.cli add-capture --project /tmp/lab/project.madrigal --pcap tests/corpus/
corpus_capture_02.pcapng
python -m src.project.cli open --path /tmp/lab/project.madrigal --show
python -m src.project.cli export --project /tmp/lab/project.madrigal --out /tmp/lab.zip
python -m src.project.cli import --in /tmp/lab.zip --target /tmp/lab-restored
```

Ожидается: `create` создаёт каталог, `add-capture` копирует файл и записывает его sha256, `open --show` печатает манифест, `export` пишет zip, а `import` распаковывает его и сообщает `"broken": []`, когда каждый захват всё ещё совпадает со своей контрольной суммой.

Проверка переносимости — суть `export`/`import`: после импорта исследование открывается и проверяется без какого-либо абсолютного пути с исходной машины.

6. Применение правила отдельно

Когда нормализованный захват уже есть, примените правило без проекта:

```
python -m src.protocol.cli apply \
  --rule src/protocol/examples/set_parameter_request.json \
  --capture out/cap01.json \
  --out out/result.json
```

Ожидается: `out/result.json` следует контракту результата: `rule_id`, `rule_version`, `capture_id`, `messages` и `summary`. Это тот же результат, что пишет конвейер, и та форма, которую читает нижележащий инструмент.

7. Интерфейс

Интерфейс PySide6 читает те же артефакты: нормализованный захват из `src.capture` и результат из `src.protocol`. Выберите сессию и направление, изучите байты с их провенансом по пакетам и откройте правило и его результаты. Отчёт и машиночитаемый результат — долговечный выход; интерфейс — просмотрщик над ними.

7.1 Язык интерфейса

Окно поставляется с английским и русским интерфейсом. Выберите язык в меню `Language`: `Russian`, `English` или `System`. `System` следует локали рабочего стола и откатывается к английскому, когда ни один каталог не совпадает. Выбор хранится через `QSettings` и восстанавливается при следующем запуске; смена языка переименовывает открытое окно сразу, без перезагрузки захвата.

Переводы лежат в `src/ui/locale/`. Английский — исходный язык, поэтому каталога не требует; русский компилируется из `madrigal_ru.ts` в `madrigal_ru.qm`, а `.qm` закоммичен, чтобы окно работало без набора инструментов Qt. После добавления или изменения строки обновите каталог:

```
scripts/update_translations.sh
```

Скрипт сканирует исходники окна, обновляет `.ts` на месте (сохраняя существующие переводы) и перекомпилирует `.qm`.

7.2 Настройки

`View -> Settings...` (или `Ctrl+,`) открывает диалог предпочтений. В нём четыре раздела: `General` для темы и языка интерфейса, `Editor` для размеров шрифта hex и дерева и колонки смещений, `Paths` для каталогов, откуда начинаются диалоги файлов, и `Advanced` для уровня логирования и заливки диагностик в hex. `OK` применяет и закрывает, `Apply` применяет без закрытия, а `Cancel` возвращает то, что было активно при открытии диалога.

Тема и язык применяются сразу через те же менеджеры, которыми пользуются меню. Значения хранятся через `QSettings` под `madrigal/protocol-lab`. Полный список ключей — в `docs/SETTINGS.md`.

8. Тесты

```
python -m pytest tests/ -q
```

Ожидается: все тесты проходят. Набор покрывает парсер, сессии, пересборку на фикстурах дефектов, согласие потокового и обычного конвейеров, круговой прогон контейнера проекта, отрисовщики отчёта и интеграцию корпуса по закоммиченным записям.

9. Установка в меню приложений

Окно поставляется со значком и ярлыком freedesktop. Установите их для текущего пользователя, без root:

```
pip install -e .
bash scripts/install_desktop.sh
```

Скрипт копирует `packaging/madrigal-protocol-lab.desktop` в `~/.local/share/applications` и отрисованные PNG в `~/.local/share/icons/hicolor/<size>x<size>/apps`, затем обновляет кэши рабочего стола и значков. Лаунчер запускает `madrigal-lab`, точку входа `gui-scripts`, объявленную в `pyproject.toml`, поэтому сначала установите пакет. Может потребоваться свежая сессия входа, прежде чем ярлык появится.

Ожидается: ярлык с именем «Madrigal Protocol Laboratory» в меню приложений, открывающий окно со значком-шестиугольником в заголовке. Мастер значка — `assets/icons/app/madrigal-protocol-lab.svg`; пересоберите PNG и их копии для hicolor через `python scripts/render_icons.py`.

Заметки

- Все команды выполняются офлайн.
- Стадии захвата, проекта и отчёта работают сами по себе. Движок правил в `src.protocol` необязателен, поэтому вариант с `--rule` в разделе 4 и отдельная команда в разделе 6 требуют этот модуль; без него конвейер всё равно работает и пишет отчёт, где сказано, что правило не применялось.
- Измеренные значения ресурсов — в `docs/BENCHMARK.md`.
- Контракты — в `docs/CONTRACT.md`; схемы JSON — в `docs/schemas/`.